

MEGN 441 – Lab 1

Step 1: Create a Node

I used the command from the lecture 4 demo -

```
ros2 pkg create demo_pubsub --build-type ament_python --node-name demo_pub
```

Only difference being I changed the name of the package and node to something more fitting

Step 2: Write the Node

Step 2.1: Getting the Imports

There are four things we need to import in order to write a node that publishes controller data to the bot:

- rclpy – ROS Client Library for Python, provides standard framework and tools.
- Node, from rclpy.node – Node is a fundamental class, is the datatype for both Publishers and Subscribers
- Twist, from geometry_msgs.msg – Twist stores 2 3D vectors for linear(x,y,z) and ang(x,y,z)
- Joy, from sensor_msgs.msg – Joy is the controller Class, it has a lot of 32 bit arrays.

Step 2.2: Writing the Class

In order to start writing a class in python, state the type, the name of the class, and whatever properties you'd want the class to have access to. An example of what we want looks as follows:

```
class ClassName(Node)
```

We want our class to be able to inherit some key functionality from the class Node. ClassName can be whatever adequate name you conjure up.

Every class has to begin with a constructor. The constructor is what initializes the object of whatever class you're trying to make. In the case of python, every constructor is named `__init__(params)`. Defining the constructor might look as follows:

```
def __init__(self)
```

self is a keyword referring to the current object (identical to "this" in c++/java)

Within the constructor, we want to make sure our node is discoverable to ROS. To do that we'll type:

```
super().__init__('node_name')
```

`super()` grabs the parent class's function of our choice, in this case, the constructor. Remember the class we're defining inherits from Node. So the line of code shown above is constructing a node, and it's titling it whatever name you throw in those quotes (hopefully not something as dull as `node_name`).

Now that we've created a space to define our constructor, we need to construct the variables to get the controller data and write controller data to the bot.

Starting with reading the controller data, we want to create something called a Subscriber that will subscribe to the /ros_robot_controller/joy topic and interpret when we want to move forward and turn. The instantiation of a subscriber looks like the following:

```
self.subscriber = self.create_subscription(  
    DataType_of_msg, 'desired_topic', self._callback, 10)
```

Where the first param describes the type of message being sent, the second contains the topic you want to subscribe to, the third is a function that we define later that runs anytime a message is sent, and the fourth being an integer dictating the size of the queue (Nave said lucky number 10 never fails)

Next, writing the control data to the bot is handled by the publisher. We want to publish the move data to the /cmd_vel topic. Instantiating a publisher is as follows:

```
self.publisher = self.create_publisher(  
    DataType_of_msg, 'desired_topic', 10)
```

The only difference between this and the subscriber's parameters is that there is no callback function needed (all the publisher does is send data to the topic).

Congrats, you've written the constructor for the class!!! Remember, we write the constructor so that objects of a class are all constructed uniformly. Without constructors, classes wouldn't even function.

Moving on from the constructor, we have to define the callback function we invoked earlier. Defining it will be similar to how we did the constructor:

```
def con_callback(self, joy_msg)
```

Note that there is a second parameter here! I called it joy_msg, but the name has zero relevance besides acting as a descriptor to us humans. joy_msg is the message we are sending from the controller to the bot.

The first step we want to take is to define a new variable that is an object of the desired class. In this case, the desired class is one that stores 2 3D vectors describing the linear and angular motion of the bot:

```
newVariable = desiredClass()
```

Now that we have the variable, we can set the forward and rotation equal to their respective joystick inputs.

```
newVariable.linear.x = joy_msg.axes[x]  
newVariable.angular.z = joy_msg.axes[y]
```

Echo the topic in ros2 to see which indices of axes should be mapped to which variable

All that's left is to upload the new data to the bot using our publisher.

```
self.publisher.publish(msg)
```

Step 2.3: Write the Main

The main's purpose is to start spinning up the Node so that an object of our class is created and operational.

```
#Standard main stuff cuz Nave said so  
rclpy.init()  
className = ClassName()  
rclpy.spin(className)  
rclpy.shutdown()
```

Step 3: Running the Code

While ssh'd into the bot, launch the launch file and run the node. Now you should be able to control the bot with the controller.